

Meet Alshi
150096724028

Github repo: <https://github.com/Meet-6226/Flutter-Assignment/tree/assignment2>

What I Learned

While doing this assignment, I learned how to use some important concepts in Dart like Null Safety, Future, async/await, mock API data, and error handling. I had used basic Dart syntax before, but I was not very clear about how asynchronous operations work. This assignment helped me understand these concepts better by actually using them in a small program.

The first thing I learned was Null Safety. In Dart, we can use `?` with a variable when it is allowed to have a null value. For example, I used `String? email` because the email may not always be available. I also learned how to check if a value is null before using it. The `??` operator was also useful because it allows us to provide a default value when the actual value is null. This made me understand how applications can handle missing data without crashing.

I also learned about the future. A Future is used when we have to wait for some data or result. This is commonly used when fetching data from an API because the response takes some time to arrive. In my program, I created a mock API using a Future instead of using a real API. The next concept I learned was `async` and `await`. I used `async` with my functions and `await` when calling the mock API. I used `Future.delayed()` to simulate the delay of a real API. This helped me understand that the program can wait for the data to be received before displaying it. I also learned how to handle errors using `try-catch`. If something goes wrong while fetching the data, the catch block can handle the error and display a proper message instead of allowing the program to stop unexpectedly.

While doing the assignment, I faced some problems understanding the difference between Future, `async`, and `await`. Initially, I was confused about why `await` was required and why the `main()` function also needed to be `async`. I also had some difficulty understanding how null values should be checked. After testing the code and making small changes, I understood these concepts better.

Overall, this assignment gave me a better understanding of how Dart handles data that takes time to load. I also learned that proper null checking and error handling are important because real applications and APIs may not always return complete or correct data.

assignment2 — main.dart

main.dart

lib > main.dart

```
22 void main() async {
25   try {
28     if (user == null) {
29       print("No user data found.");
30     } else {
31       print("User data found!");
32
33       print("ID: ${user.id}");
34       print("Name: ${user.name}");
35
36       if (user.email == null) {
37         print("Email: Email not available");
38       } else {
39         print("Email: ${user.email}");
40       }
41     }
42   } catch (e) {
```

Problems

Output

Debug Console

Terminal

Ports

meetaishi@Mac assignment2 % flutter run ./lib/main.dart

r Hot reload. 📱🔥

R Hot restart.

h List all available interactive commands.

d Detach (terminate "flutter run" but leave application running).

c Clear the screen

q Quit (terminate the application on the device).

This app is linked to the debug service: ws://127.0.0.1:59272/S17KPFaoeIo=/ws

Debug service listening on ws://127.0.0.1:59272/S17KPFaoeIo=/ws

A Dart VM Service on Chrome is available at: http://127.0.0.1:59272/S17KPFaoeIo=

The Flutter DevTools debugger and profiler on Chrome is available at: http://127.0.0.1:59272/S17KPFaoeIo=/devtools/?uri=ws://127.0.0.1:59272/S17KPFaoeIo=/ws

Starting application from main method in: org-dartlang-app:///web_entrypoint.dart.

Fetching user data...

A message on the flutter/lifecycle channel was discarded before it could be handled. This happens when a plugin sends messages to the framework side before the framework has had an opportunity to register a listener. See the ChannelBuffers API documentation for details on how to configure the channel to expect more messages, or to expect messages to get discarded: https://api.flutter.dev/flutter/dart-ui/ChannelBuffers-class.html

User data found!

ID: 1

Name: Meet

Email: Email not available

Outline

Timeline

Java Projects